



深圳大学
Shenzhen University

互联网编程

第11章

非阻塞IO

深圳大学计算机与软件学院

概述

- 非阻塞I/O
 - 字节流InputStream、OutputStream的阻塞I/O
 - 非阻塞是不是一定比阻塞好？
- 允许CPU速度高于网络的方案：
 - 缓冲+多线程
 - 线程切换
 - 多核
- NIO的优劣势
 - 适用于维持大量长期但不非常活跃的连接
 - 如推送服务的后台
 - 但增加了代码复杂度：不要贸然优化
- Java.nio包



一个基于通道的chargen客户端示例

■ 通道Channel

- 类似Stream，通过它向数据源读（或目的地写）数据块（Buffer）
 - 区别：Stream读写的是一个字节，Channel读写的是一个数据块
- 提供I/O异步支持
- <http://docs.oracle.com/javase/8/docs/api/java/nio/channels/Channel.html>

■ SocketChannel

- <http://docs.oracle.com/javase/8/docs/api/java/nio/channels/SocketChannel.html>

■ 缓冲区Buffer

- 通常是一个字节数组 + 几个Index（ $\text{mark} \leq \text{position} \leq \text{limit} \leq \text{capacity}$ ）
- 相对和绝对的get、put
- flip & rewind
- <http://docs.oracle.com/javase/8/docs/api/java/nio/Buffer.html>



一个基于通道的chargen客户端示例

EX11-1 ChargenClient chargen（无限循环）协议客户端。Ctrl-C结束程序

协议内容：服务器在端口19监听连接，当客户端连接时，服务器发送连续的字符序列，直到客户端断开连接为止。客户端所有输入都被忽略。以回车/换行作为行分隔符的72字符循环文本行

ChargenClient.java

```
1 import java.nio.*;
2 import java.nio.channels.*;
3 import java.net.*;
4 import java.io.IOException;
5
6 public class ChargenClient {
7
8     public static int DEFAULT_PORT = 19;
9
10    public static void main(String[] args) {
11
12        if (args.length == 0) {
13            System.out.println("Usage: java ChargenClient host [port]");
14            return;
15        }
16
17        int port;
18        try {
19            port = Integer.parseInt(args[1]);
20        } catch (RuntimeException ex) {
21            port = DEFAULT_PORT;
22        }
23
24        try {
25            SocketAddress address = new InetSocketAddress(args[0], port);
26            SocketChannel client = SocketChannel.open(address);
```

一个基于通道的chargen客户端示例

ChargenClient.java

```
27
28     ByteBuffer buffer = ByteBuffer.allocate(74);
29     WritableByteChannel out = Channels.newChannel(System.out);
30
31     while (client.read(buffer) != -1) {
32         buffer.flip();
33         out.write(buffer);
34         buffer.clear();
35     }
36 } catch (IOException ex) {
37     ex.printStackTrace();
38 }
39 }
40 }
```



一个非阻塞的changen服务器示例

■ 选择器

- 作用于多个SelectableChannel上的一个控制选择器
- SelectableChannel向选择器注册（register）时得到的指针，选择器工作（选择）时，将返回符合条件的这些指针的集合
 - 选择器维持：key set、selected-key set和cancelled-key set三个集合
 - selected-key中的key，通常处理后需要remove（通过set或iterator）
- 选择：
 - 阻塞型选择select(), select(long)和非阻塞型选择selectNow()
 - 查询操作系统，询问已注册的这些Channel，他们感兴趣的事情是否已经发生、就绪；若是把对应的key加入selected-key set中
- 并发
 - 选择器本身是线程安全的，但三个key set不是

☞ <http://docs.oracle.com/javase/8/docs/api/java/nio/channels/Selector.html>



一个非阻塞的chargen服务器示例

- 创建ServerSocketChannel
- 获得ServerSocket并绑定端口
- （可选）服务端非阻塞的accept模式
 - 配置serverChannel采用非阻塞模式
 - 构造selector并进行accept ready的注册
 - selector开始工作（select），选出已经accept的channel的key
 - 对选出来的accept ready的key，获得channel，accept得到SocketChannel，然后配置buffer，进行非阻塞的IO
 - 特别的remove处理



一个非阻塞的chargen服务器示例

EX11-2 ChargenServer, 一个非阻塞的chargen服务器

ChargenServer.java

```
1 import java.nio.*;
2 import java.nio.channels.*;
3 import java.net.*;
4 import java.util.*;
5 import java.io.IOException;
6
7 public class ChargenServer {
8
9     public static int DEFAULT_PORT = 19;
10
11     public static void main(String[] args) {
12
13         int port;
14         try {
15             port = Integer.parseInt(args[0]);
16         } catch (RuntimeException ex) {
17             port = DEFAULT_PORT;
18         }
19         System.out.println("Listening for connections on port " + port);
20
21         byte[] rotation = new byte[95*2];
22         for (byte i = ' '; i <= '~'; i++) {
23             rotation[i - ' '] = i;
24             rotation[i + 95 - ' '] = i;
25         }
```


一个非阻塞的chargen服务器示例

ChargenServer.java

```
26
27 ServerSocketChannel serverChannel;
28 Selector selector;
29 try {
30     serverChannel = ServerSocketChannel.open();
31     ServerSocket ss = serverChannel.socket();
32     InetSocketAddress address = new InetSocketAddress(port);
33     ss.bind(address);
34     serverChannel.configureBlocking(false);
35     selector = Selector.open();
36     serverChannel.register(selector, SelectionKey.OP_ACCEPT);
37 } catch (IOException ex) {
38     ex.printStackTrace();
39     return;
40 }
41
42 while (true) {
43     try {
44         selector.select();
45     } catch (IOException ex) {
46         ex.printStackTrace();
47         break;
48     }
49
50     Set<SelectionKey> readyKeys = selector.selectedKeys();
51     Iterator<SelectionKey> iterator = readyKeys.iterator();
52     while (iterator.hasNext()) {
```



```
53
54     SelectionKey key = iterator.next();
55     iterator.remove();
56     try {
57         if (key.isAcceptable()) {
58             ServerSocketChannel server = (ServerSocketChannel) key.channel();
59             SocketChannel client = server.accept();
60             System.out.println("Accepted connection from " + client);
61             client.configureBlocking(false);
62             SelectionKey key2 = client.register(selector, SelectionKey.
63                                                         OP_WRITE);
64             ByteBuffer buffer = ByteBuffer.allocate(74);
65             buffer.put(rotation, 0, 72);
66             buffer.put((byte) '\r');
67             buffer.put((byte) '\n');
68             buffer.flip();
69             key2.attach(buffer);
70         } else if (key.isWritable()) {
71             SocketChannel client = (SocketChannel) key.channel();
72             ByteBuffer buffer = (ByteBuffer) key.attachment();
73             if (!buffer.hasRemaining()) {
74                 // Refill the buffer with the next line
75                 buffer.rewind();
76                 // Get the old first character
77                 int first = buffer.get();
78                 // Get ready to change the data in the buffer
79                 buffer.rewind();
```

```
80         // Find the new first characters position in rotation
81         int position = first - ' ' + 1;
82         // copy the data from rotation into the buffer
83         buffer.put(rotation, position, 72);
84         // Store a line break at the end of the buffer
85         buffer.put((byte) '\r');
86         buffer.put((byte) '\n');
87         // Prepare the buffer for writing
88         buffer.flip();
89     }
90     client.write(buffer);
91 }
92 } catch (IOException ex) {
93     key.cancel();
94     try {
95         key.channel().close();
96     }
97     catch (IOException cex) {}
98 }
99 }
100 }
101 }
102 }
```

缓冲区

- 创建缓冲区
 - 分配
 - 直接分配
 - 包装
- 填充和排空
- 批量方法
- 数据转换
- 视图缓冲区
- 压缩缓冲区
 - 剩余数据移到开头
- 复制缓冲区
- 分片缓冲区
- 标记和重置
- Object方法



通道

- Channels类是一个工具类，可将基于I/O的流、阅读器和书写器包装在通道中，也可以从通道转换为基于I/O的流、阅读器和书写器。
- SocketChannel类实现了Channels类中的Readable(Writable)ByteChannel接口
 - ❧ 连接
 - ❧ 读取
 - ❧ 写入
 - ❧ 关闭
- ServerSocketChannel
 - ❧ 创建服务器Socket通道
 - ❧ 接受连接
- 异步通道（Java7及以上）AsynchronousSocketChannel类和AsynchronousServerSocketChannel类



就绪选择

- 即能够选择读写时不阻塞的socket
- 进行就绪选择，要将不同的通道注册到一个selector对象，每个通道分配一个selectionKey，需：
- Selector类
- SelectionKey类

